

โครงการ

Object-Oriented Programming
Mini Project เกม Sushi Rush

จัดทำโดย

6804062610126 นายกฤษณะ เอื้อราษฎร์

เสนอ

ผู้ช่วยศาสตราจารย์สถิต ประสมพันธ์
วิชา 040613204 Object-Oriented Programming
ภาคเรียนที่ 1 ปีการศึกษา 2569

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ คณะวิทยาศาสตร์ประยุกต์
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

สารบัญ

เรื่อง	หน้า
บทที่ 1 บทนำ	1
1.1 ที่มาและความสำคัญของโปรเจ็ค	1
1.2 ประเภทของโครงการ	1
1.3 ประโยชน์ที่ได้รับ	1
1.4 ขอบเขตของโครงการ	1
บทที่ 2 ส่วนการพัฒนา	2
2.1 เนื้อเรื่องย่อหรือวิธีการเล่น	2
2.2 คลาสไดอะแกรม พร้อมคำอธิบาย	4
2.3 รูปแบบการพัฒนา Application / Applet	5
2.4 อธิบายส่วนของโปรแกรมที่มีหลักการ OOP	5
2.5 หน้าจอ GUI อธิบายส่วนประกอบของ GUI	8
2.6 อธิบาย Event handling ที่มีในหน้าจอ	9
2.7 อธิบายอัลกอริทึมที่สำคัญในโปรแกรม	9
บทที่ 3 สรุปผล	11
3.1 ปัญหาที่พบระหว่างการพัฒนา	11
3.2 จุดเด่นของโปรแกรมที่ไม่เหมือนใคร	11
3.3 คำแนะนำสำหรับผู้สอน	12

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญของโปรเจ็ค

ในการศึกษาการเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming) การทำความเข้าใจผ่านทฤษฎีเพียงอย่างเดียวอาจไม่เพียงพอ ผู้จัดทำจึงมีแนวคิดที่จะนำหลักการ OOP ทั้งหมดมาประยุกต์ใช้ในการสร้างโปรแกรมที่สามารถโต้ตอบกับผู้ใช้งานได้จริงแบบเรียลไทม์ ซึ่ง 'เกม' เป็นซอฟต์แวร์ที่ตอบโจทย์นี้ได้ดีที่สุด โครงงานเกม Sushi Rush จึงถูกพัฒนาขึ้นด้วยภาษา Java (ผ่านไลบรารี Swing/AWT) เพื่อใช้เป็นสนามทดลองในการออกแบบโครงสร้างคลาส (Class Design) การจัดการอินพุตจากผู้ใช้ (Event Handling) และการวาดภาพกราฟิก ซึ่งช่วยเชื่อมโยงความรู้ทางทฤษฎีสู่การปฏิบัติจริงได้อย่างเป็นรูปธรรม

1.2 ประเภทของโครงการ

โปรแกรมประยุกต์ประเภทเกมคอมพิวเตอร์ 2 มิติ (2D Desktop Application Game) แนวบริหารเวลาและจำลองสถานการณ์ (Time-Management)

1.3 ประโยชน์ที่ได้รับ

1. ด้านการประยุกต์ใช้ความรู้: ได้นำหลักการ OOP ทั้ง Encapsulation, Inheritance, Polymorphism, Abstraction และ Composition มาใช้งานจริงในการออกแบบคลาสต่างๆ ของเกม (เช่น คลาสตัวละคร, คลาสซูชิ)
2. ด้านทักษะการแก้ปัญหา: ได้เรียนรู้วิธีการเขียน Game Loop, การคำนวณเวลาเฟรมเรต (Delta Time) และการสร้างระบบ Hitbox เพื่อตรวจจับพิกัดการลากวาง (Drag & Drop) แบบ Custom ด้วยตัวเอง
3. ด้านผู้ใช้งาน: โปรแกรมสามารถสร้างความบันเทิง และช่วยฝึกทักษะการตัดสินใจ การบริหารเวลา และความแม่นยำให้กับผู้เล่นผ่านระบบเกมเพลย์ที่ท้าทาย

1.4 ขอบเขตของโครงการ

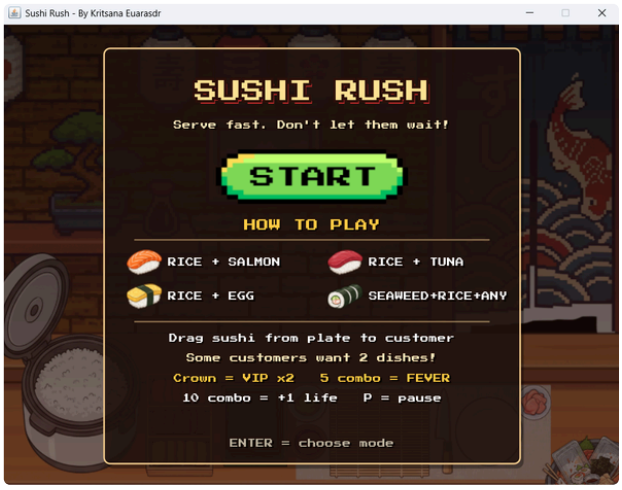
1. ตัวเกมพัฒนาด้วยภาษา Java และประมวลผลกราฟิกผ่าน Java Swing/AWT
2. รองรับลูกค้าบนโต๊ะบาร์สูงสุด 3 คนพร้อมกัน โดยแต่ละคนมีหลอดความอดทนและอารมณ์แตกต่างกัน
3. มีระบบผสมวัตถุดิบซูชิ และใช้เมาส์ในการลากเสิร์ฟ (Drag & Drop) ไปยังตัวลูกค้า
4. มีระบบคำนวณคะแนนพิเศษ (Combo System) และระบบแอนิเมชัน Particle Effect

บทที่ 2 ส่วนการพัฒนา

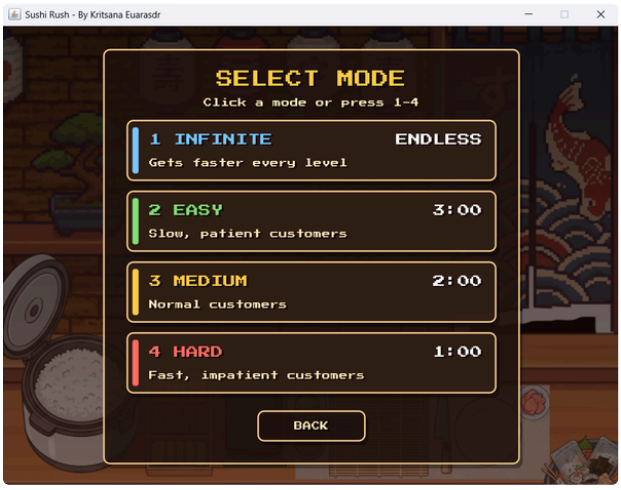
2.1 เนื้อเรื่องย่อหรือวิธีการเล่น

ผู้เล่นจะรับบทเป็นเชฟร้านซูชิที่จะต้องเลือกโหมดการเล่น (GameMode) และคอยรับออเดอร์จากลูกค้าที่สุ่มเดินเข้ามาในร้าน (สูงสุด 3 คนพร้อมกัน) ซึ่งมีทั้งลูกค้าทั่วไปและลูกค้า VIP (VipCustomer) ที่มีหลอดความอดทน (Patience) แตกต่างกัน ผู้เล่นต้องใช้เมาส์คลิกส่วนผสมบนโต๊ะ ได้แก่ ข้าว, สาหร่าย และหน้าซูชิ (Salmon, Tuna, Tamago) เพื่อประกอบเป็นเมนูที่ถูกต้อง เมื่อทำเสร็จต้องลาก (Drag & Drop) ซูชิไปเสิร์ฟให้ตรงกับออเดอร์ของลูกค้า หากเสิร์ฟถูกต้องจะได้คะแนนและโบนัสคอมโบต่อเนื่อง (Combo System) พร้อมเอฟเฟกต์พิเศษ แต่หากเสิร์ฟผิดหรือปล่อยให้ลูกค้ารอนานจนหลอดความอดทนหมด ลูกค้าจะโกรธและเดินออกจากร้าน ทำให้สูญเสียพลังชีวิต (Lives) เมื่อพลังชีวิตเหลือศูนย์เกมจะจบลง (Game Over) และระบบจะทำการตรวจสอบเพื่อบันทึกสถิติคะแนนสูงสุด (HighScore)

เมื่อเปิดเกมขึ้นมาจะพบกับหน้าเมนูหลัก ซึ่งมีปุ่ม START และสรุปสูตรซูชิไว้ให้ผู้เล่นดูก่อนเริ่ม เมื่อกดปุ่ม START หรือปุ่ม Enter จะเข้าสู่หน้าเลือกโหมด ซึ่งมีทั้งหมด 4 โหมด ได้แก่ โหมด INFINITE ที่ไม่จำกัดเวลา เล่นไปจนกว่าชีวิตจะหมด และลูกค้าจะเร็วขึ้นทุกเลเวล โหมด EASY มีเวลา 3 นาที ลูกค้าเดินช้าและรอได้นาน โหมด MEDIUM มีเวลา 2 นาที ลูกค้ามีความเร็วปกติ และโหมด HARD มีเวลา 1 นาที ลูกค้าเดินเร็ว ใจร้อน และสั่งหลายจานบ่อยกว่าโหมดอื่น ผู้เล่นเลือกโหมดได้ด้วยการคลิก หรือกดเลข 1–4 บนคีย์บอร์ด ทั้งนี้แต่ละโหมดจะเก็บคะแนนสูงสุดแยกกัน



ภาพประกอบ หน้าเริ่มเกม



ภาพประกอบ หน้าเลือกโหมดการเล่น

ในหน้าจอระหว่างเล่น ส่วนบนของจอแสดงคะแนน จำนวนชีวิต (เริ่มต้น 3 ดวง) เลเวล เวลาที่เหลือ และคอมโบ ส่วนกลางเป็นที่ยืนของลูกค้า ซึ่งมีได้พร้อมกันสูงสุด 3 คน แต่ละคนมีใบสั่งอาหารและหลอดความอดทนอยู่เหนือศีรษะ ส่วนล่างเป็นโต๊ะทำซูชิ ประกอบด้วยหม้อข้าว ถาดหน้าซูชิ (แซลมอน ทูน่า ไข่หวาน) สาหร่าย เสื่อม้วนซูชิ จานเสิร์ฟ และถังขยะ

บทที่ 2 ส่วนการพัฒนา



ภาพประกอบ หน้าจอขณะเล่นเกม



ภาพประกอบ ประเภทของลูกค้า

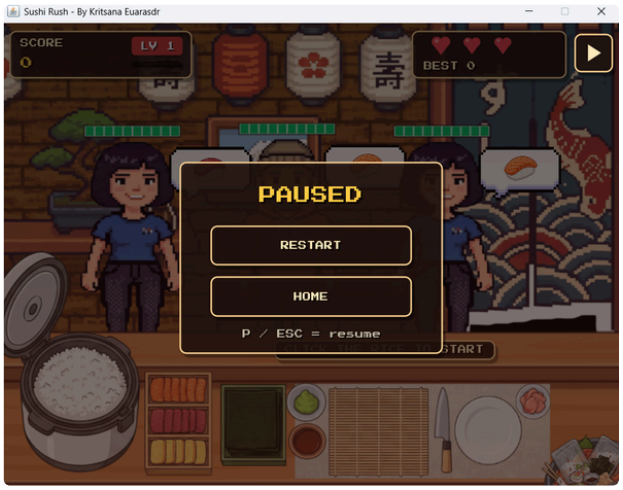
ผู้เล่นทำซูชิได้โดยคลิกวัตถุดิบบนโต๊ะตามลำดับ ซูชิแบบนิกิริทำได้โดยคลิกข้าวแล้วตามด้วยหน้าที่ต้องการ คือ ข้าว + แซลมอน ได้ Salmon Nigiri ข้าว + ทูน่า ได้ Tuna Nigiri และข้าว + ไข่หวาน ได้ Tamago Nigiri ส่วน Maki Roll ต้องวางสาหร่ายก่อน แล้วจึงใส่ข้าวและหน้าใดก็ได้ ทั้งนี้ต้องใส่ข้าวก่อนหน้าซูชิเสมอ และทำได้ครั้งละหนึ่งจาน หากใส่วัตถุดิบผิดสามารถคลิกถังขยะเพื่อทิ้งแล้วเริ่มใหม่ได้ เมื่อทำซูชิเสร็จ ซูชิจะปรากฏบนจานพร้อมแสงกะพริบ ผู้เล่นต้องกดค้างที่ซูชิแล้วลากไปปล่อยที่ลูกค้าที่สั่งเมนูนั้น (Drag & Drop)

หากเสิร์ฟถูกต้อง ลูกค้าจะดีใจ และผู้เล่นจะได้คะแนนพื้นฐาน 100 คะแนน บวกโบนัสความเร็วสูงสุด 50 คะแนนตามความอดทนที่ลูกค้ายังเหลือ และโบนัสคอมโบอีก 15 คะแนนต่อคอมโบ หากเสิร์ฟผิด หรือปล่อยให้หลอดความอดทนของลูกค้าหมด ลูกค้าจะโกรธและเดินออกจากร้าน ผู้เล่นจะเสียชีวิต 1 ดวง และคอมโบจะกลับเป็นศูนย์ ลูกค้าบางคนสั่งอาหาร 2 จาน หรือขอสั่งเพิ่มหลังได้รับครบแล้ว และตั้งแต่เลเวล 2 เป็นต้นไปจะมีลูกค้า VIP ซึ่งสวมมงกุฎและมีแสงสีทอง ลูกค้า VIP จะใจร้อนกว่าลูกค้าปกติ แต่ให้คะแนนเป็น 2 เท่า

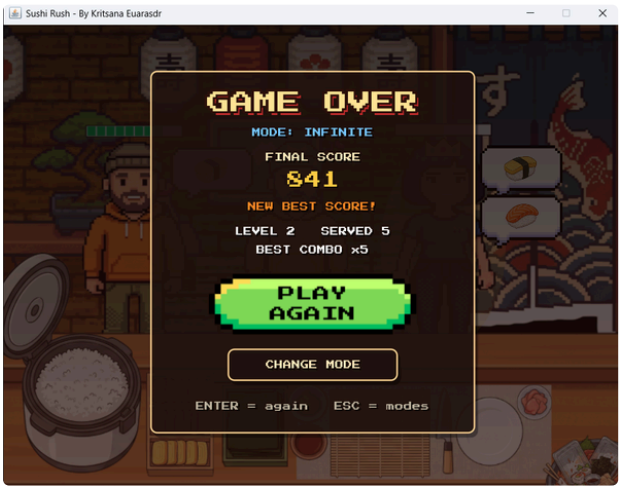
เมื่อเสิร์ฟถูกต่อเนื่องครบทุก 5 คอมโบ จะเข้าสู่ช่วง FEVER TIME นาน 8 วินาที ทุกจานที่เสิร์ฟในช่วงนี้ได้คะแนน 2 เท่า และเมื่อครบทุก 10 คอมโบ จะได้ชีวิตคืน 1 ดวง นอกจากนี้ เมื่อเสิร์ฟถูกครบทุก 5 จาน เกมจะขึ้นเลเวลใหม่ ลูกค้าจะรอได้สั้นลงและสั่งหลายจานบ่อยขึ้น ระหว่างเล่นผู้เล่นสามารถกดปุ่ม P หรือปุ่มหยุดที่มุมขวาบนเพื่อหยุดเกมชั่วคราว แล้วเลือกเล่นต่อ เริ่มใหม่ หรือกลับหน้าหลักได้

เกมจะจบเมื่อชีวิตหมดทั้ง 3 ดวง หรือหมดเวลาในโหมดที่จำกัดเวลา หน้า Game Over จะแสดงคะแนน เลเวล และจำนวนจานที่เสิร์ฟได้ หากคะแนนสูงกว่าสถิติเดิม ระบบจะบันทึกเป็นคะแนนสูงสุดใหม่ (High Score) ลงไฟล์ ปิดเกมแล้วเปิดใหม่คะแนนก็ยังอยู่ จากนั้นผู้เล่นสามารถกด PLAY AGAIN เพื่อเล่นโหมดเดิมอีกครั้ง หรือกด CHANGE MODE เพื่อเลือกโหมดใหม่

บทที่ 2 ส่วนการพัฒนา

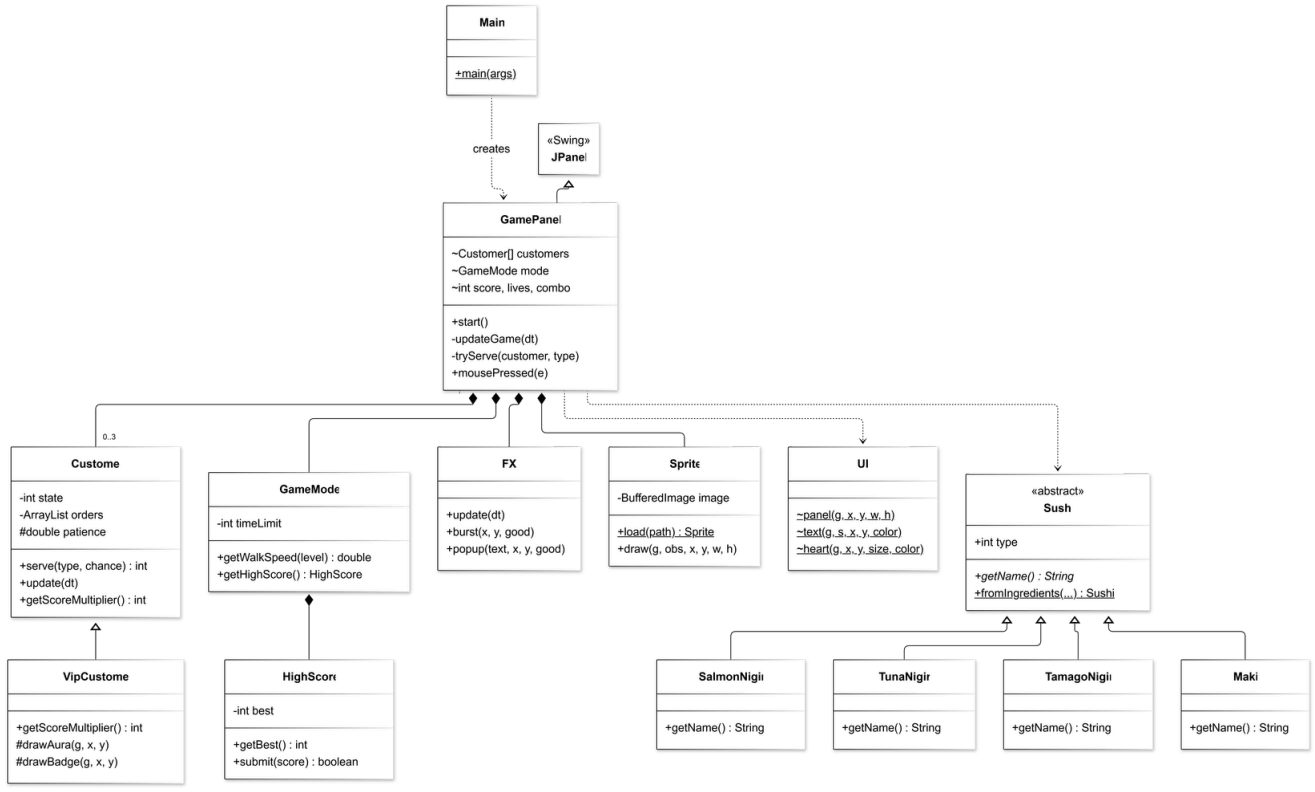


ภาพประกอบ หน้าหยุดเกมชั่วคราว



ภาพประกอบ หน้า Game Over

2.2 คลาสไดอะแกรม พร้อมคำอธิบาย



ภาพประกอบ คลาสไดอะแกรม

บทที่ 2 ส่วนการพัฒนา

1. **Main** คือ คลาสแกนหลัก ทำหน้าที่สร้างหน้าต่างโปรแกรม JFrame และเรียกใช้งาน GamePanel
2. **GamePanel** คือ คลาสควบคุมหลัก (สืบทอดจาก JPanel) จัดการสถานะของเกม (State Machine), วาดหน้าจอ GUI, ตรวจจับเมาส์ และควบคุมลูปเกม (Game Loop)
3. **Customer** คือ คลาสจัดการข้อมูลลูกค้า เช่น พิกัด, หลอดความอดทน, ชนิดอาหารที่สั่ง และการคำนวณแอนิเมชันการเคลื่อนไหว
4. **Sushi** (Abstract Class) คือ คลาสแม่แบบของเมนูซูชิ มีคลาสย่อยที่สืบทอดไปใช้งาน ได้แก่ SalmonNigiri, TunaNigiri, TamagoNigiri และ Maki
5. **Sprite** คือ คลาสอำนวยความสะดวกในการอ่านไฟล์ภาพ (ImageIO) และวาดภาพลงบนหน้าจอ
6. **FX** คือ คลาสจัดการระบบอนุภาค (Particle System) เช่น แอนิเมชันตัวหนังสือลอย และเอฟเฟกต์พลุกระจาย
7. **VipCustomer** คือ คลาสลูกค้าพิเศษที่สืบทอดคุณสมบัติ (Inheritance) มาจากคลาส Customer มีความอดทนน้อยกว่าลูกค้าปกติ 25% แต่ให้คะแนนเป็น 2 เท่าเมื่อเสิร์ฟสำเร็จ และมีการวาดมงกุฎกับแสงสีทองเพื่อให้แยกออกจากลูกค้าทั่วไปได้
8. **UI** คือ คลาสจัดการการวาดส่วนติดต่อผู้ใช้งาน (User Interface) หน้าจอเมนูหลัก หน้าจอเลือกโหมด แถบแสดงสถานะ (HUD) และหน้าจอสรุปผลเมื่อจบเกม รวมไปถึงการโหลด Custom Font (เช่น ฟอนต์ตระกูล Pixel Art)
9. **GameMode** คือ คลาสจัดการโหมดการเล่นและระดับความยากของเกม ใช้กำหนดค่าพารามิเตอร์ของแต่ละโหมด ได้แก่ เวลาในการเล่น ความเร็วในการเดินของลูกค้า ความเร็วในการลดลงของหลอดความอดทน และโอกาสที่ลูกค้าจะสั่งอาหาร 2 จานหรือสั่งเพิ่ม รวมถึงเก็บคะแนนสูงสุดแยกของแต่ละโหมด
10. **HighScore** คือ คลาสจัดการระบบสถิติคะแนนสูงสุด (Best Score) ทำหน้าที่ตรวจสอบ อ่าน และบันทึกคะแนนสูงสุดของผู้เล่น

2.3 รูปแบบการพัฒนา Application / Applet

พัฒนาเป็น Java Desktop Application โดยใช้งานไลบรารีพื้นฐานของ Java ได้แก่ Java Swing (JFrame, JPanel) สำหรับสร้างโครงสร้างหน้าต่างโปรแกรม และ Java AWT (Graphics2D, BufferedImage) สำหรับการประมวลผลกราฟิก 2 มิติ ส่วนการควบคุมเวลาของเกมใช้ Thread แยกทำหน้าที่เป็น Game Loop

2.4 อธิบายส่วนของโปรแกรมที่มีหลักการ OOP

1. **Constructor** นำมาใช้กำหนดค่าเริ่มต้นให้กับวัตถุทุกครั้งที่ถูกสร้าง เช่น ในคลาส Customer จะรับพิกัด X, Y สุ่มเลือกสกินตัวละคร และคำนวณหลอดเวลาเริ่มต้นผ่าน Constructor ทันทีที่ลูกค้าเข้าร้าน

```
public Customer(int slot, int targetX, int standY, double patienceSeconds, double walkSpeed,
        double patienceDrain, int orderCount,
        Sprite[] girl, Sprite[] guy, Sprite[] bubbles, Sprite[] bars) {
    this.slot = slot;
    this.targetX = targetX;
    this.x = DOOR_X;
    this.y = standY;
    this.standY = standY;
    this.walkSpeed = walkSpeed;
    this.patienceDrain = patienceDrain;
    this.state = WALKING_IN;
    this.reactionTime = 0;
    this.timedOut = false;
    this.hasReordered = false;
    this.orderBubbles = bubbles;
```

2. **Encapsulation** มีการซ่อนข้อมูลสำคัญด้วย private เช่น ตัวแปร state และ orders ในคลาส Customer และตัวแปร best ในคลาส HighScore ซึ่งคลาสอื่นจะแก้ไขโดยตรงไม่ได้ ต้องเปลี่ยนแปลงผ่านเมธอดภายในคลาส เช่น serve() และ submit() เท่านั้น

```
public class HighScore {
    private final File file;
    private int best;

    public HighScore(String path) {
        this.file = new File(path);
        this.best = load();
    }

    public int getBest() {
        return best;
    }

    // คืน true ถ้าคะแนนนี้ทำลายสถิติเดิม
    public boolean submit(int score) {
        if (score <= best) {
            return false;
        }
        best = score;
        save();
        return true;
    }
```

3. **Composition** การประกอบออบเจกต์ (Has-a relationship) โดยคลาส GamePanel มีอาร์เรย์ของ Customer และออบเจกต์ของคลาส GameMode, FX และ Sprite เป็นองค์ประกอบ ส่วนคลาส GameMode ก็มีออบเจกต์ HighScore อยู่ภายในเพื่อเก็บคะแนนสูงสุดของแต่ละโหมด

```
    private final HighScore highScore;
    private final Color color;

    public GameMode(String name, String description, int timeLimit,
            double walkSpeed, double walkSpeedPerLevel,
            double patienceDrain, double patienceDrainPerLevel,
            double doubleOrderChance, double reorderChance,
            String scoreFile, Color color) {
        this.name = name;
        this.description = description;
        this.timeLimit = timeLimit;
        this.walkSpeed = walkSpeed;
        this.walkSpeedPerLevel = walkSpeedPerLevel;
        this.patienceDrain = patienceDrain;
        this.patienceDrainPerLevel = patienceDrainPerLevel;
        this.doubleOrderChance = doubleOrderChance;
        this.reorderChance = reorderChance;
        this.highScore = new HighScore(scoreFile);
        this.color = color;
    }
```

4. **Abstract** การประกาศ public abstract class Sushi เป็นคลาสแม่แบบส่วนกลาง ซึ่งโปรแกรมจะไม่สามารถสร้างออบเจกต์จากคลาส Sushi โดยตรงได้ (ไม่สามารถ new Sushi()) แต่จะต้องสร้างผ่านคลาสย่อยที่มีการระบุหน้าซูชิชัดเจนแล้วเท่านั้น

```java
public abstract class Sushi {
    public static final int NONE = -1;
    public static final int SALMON = 0;
    public static final int TUNA = 1;
    public static final int TAMAGO = 2;
    public static final int MAKI = 3;

    public final int type;

    protected Sushi(int type) {
        this.type = type;
    }

    // ซูชิแต่ละชนิดบอกชื่อตัวเอง (polymorphism)
    public abstract String getName();
```

5. **Polymorphism** มีการประยุกต์ใช้ Polymorphism 2 ส่วนหลัก ได้แก่

- **ฝั่งเมนูซูชิ** เมธอด Sushi.fromIngredients() สามารถส่งคืนค่ากลับมาเป็นออบเจกต์ของคลาสลูกชนิดใดก็ได้ (SalmonNigiri, TunaNigiri, TamagoNigiri, Maki) โดยอ้างอิงผ่านชนิดข้อมูลของคลาสแม่คือ Sushi และเมื่อเรียกเมธอด getName() แต่ละคลาสจะคืนชื่อเมนูของตัวเอง
- **ฝั่งตัวละครลูกค้า** อาร์เรย์ Customer[] customers ใน GamePanel สามารถเก็บได้ทั้งออบเจกต์ของ Customer ทั่วไปและ VipCustomer เมื่อโปรแกรมเรียกเมธอด getScoreMultiplier(), drawAura() หรือ drawBadge() จะเลือกทำงานตามคลาสจริงของออบเจกต์ที่ถูก Override ไว้โดยอัตโนมัติ (Dynamic Method Dispatch) เช่น VipCustomer จะคืนตัวคูณคะแนนเป็น 2 และวาดมงกุฎกับแสงสีทองเพิ่ม

```java
    private void spawnInSlot(int slot) {
        int orderCount = rand.nextDouble() < doubleOrderChance() ? 2 : 1;
        double patience = currentPatience();
        if (orderCount == 2) {
            patience = patience * 1.6;
        }
        double speed = mode.getWalkSpeed(level);
        double drain = mode.getPatienceDrain(level);
        if (rand.nextDouble() < vipChance()) {
            customers[slot] = new VipCustomer(slot, SLOT_X[slot], STAND_Y, patience, speed, drain,
                    orderCount, girlSprites, guySprites, orderBubbles, patienceBars);
        } else {
            customers[slot] = new Customer(slot, SLOT_X[slot], STAND_Y, patience, speed, drain,
                    orderCount, girlSprites, guySprites, orderBubbles, patienceBars);
        }
    }
```

6. **Inheritance** การสืบทอดคุณสมบัติ โดยคลาส SalmonNigiri, TunaNigiri, TamagoNigiri และ Maki สืบทอดมาจากคลาส Sushi, คลาส VipCustomer สืบทอดมาจากคลาส Customer เพื่อขยายความสามารถของลูกค้าพิเศษ และคลาส GamePanel ที่สืบทอดมาจาก JPanel

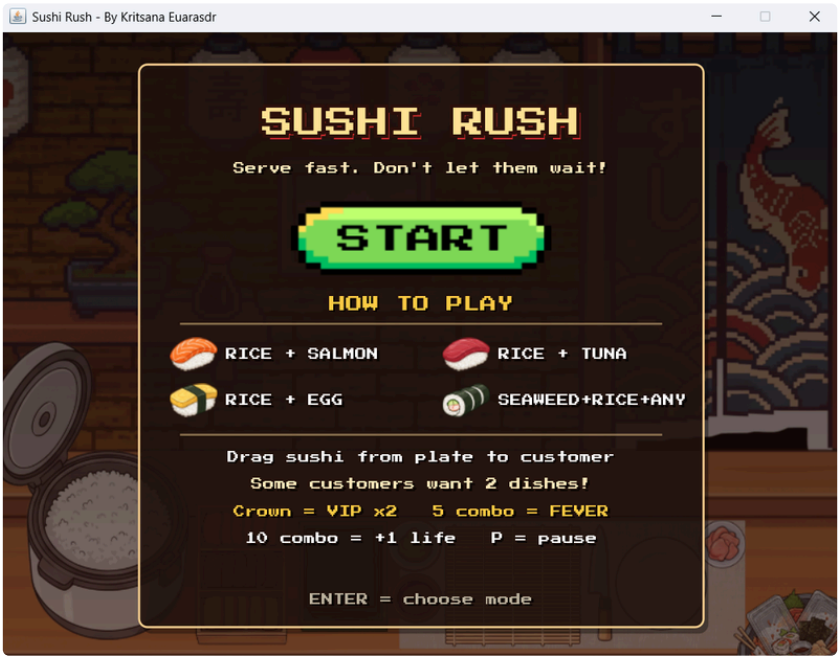
```java
class SalmonNigiri extends Sushi {
    SalmonNigiri() {
        super(SALMON);
    }

    @Override
    public String getName() {
        return "SALMON NIGIRI";
    }
}
```

บทที่ 2 ส่วนการพัฒนา

2.5 หน้าจอ GUI อธิบายส่วนประกอบของ GUI

โครงสร้างของโปรแกรมใช้ JFrame เป็นหน้าต่างหลัก และมี GamePanel (JPanel) เป็นพื้นที่แสดงผล โดยใช้เทคนิค Custom Rendering แยกการจัดการหน้าจอไว้ที่คลาส UI ร่วมกับคลาส Sprite และเมธอด Graphics2D.drawImage ในการวาดรูปภาพพื้นหลัง อุปกรณ์บนโต๊ะ หน้าจอเมนู และ HUD ขึ้นมาเองทั้งหมด พร้อมทั้งโหลดฟอนต์สไตล์ Pixel Art (PressStart2P) มาแสดงผลข้อความในเกมให้คุมโทนเดียวกัน จากนั้นจำลองการทำงานของปุ่มและวัตถุโดยการเช็คพิกัด Hitbox จากเมาส์





บทที่ 2 ส่วนการพัฒนา

2.6 อธิบาย Event handling ที่มีในหน้าจอ

1. MouseListener

   1.1 mousePressed ดักจับจังหวะการกดเมาส์ลง เพื่อตรวจจับว่าผู้เล่นคลิกโดนวัตถุดิบชิ้นใดบนโต๊ะ (ผ่านการคำนวณพิกัด X, Y) และใช้ตรวจจับการเริ่มหยิบจานซูชิ

   1.2 mouseReleased ทำงานร่วมกับระบบ Drag & Drop เมื่อผู้เล่นปล่อยเมาส์ ระบบจะดึงพิกัดสุดท้ายมาตรวจสอบว่าตรงกับ Hitbox ของลูกค้าคนใดหรือไม่ เพื่อเรียกใช้เมธอดการเสิร์ฟอาหาร

2. **MouseMotionListener** (mouseDragged) ดักจับเหตุการณ์ขณะลากเมาส์ เพื่ออัปเดตพิกัดแกน X, Y ของเมาส์แบบเรียลไทม์ ทำให้ภาพซูชิบนจานสามารถลอยตามเคอร์เซอร์ของเมาส์ได้อย่างสมูท
3. **Game** Loop (Thread) ใช้ Thread แยกทำงานวนรอบ 60 ครั้งต่อวินาที (60 FPS) โดยในแต่ละรอบจะเรียก updateGame() เพื่ออัปเดตสถานะเกม แล้ววาดหน้าจอใหม่ด้วย paintImmediately() ผ่าน SwingUtilities.invokeAndWait() เพื่อให้ทำงานบน Swing thread อย่างปลอดภัย
4. **KeyListener (keyPressed)** ดักจับการกดคีย์บอร์ด ได้แก่ Enter/Space เพื่อเริ่มเกม ปุ่ม 1–4 เพื่อเลือกโหมด และ P/Esc เพื่อหยุดเกมชั่วคราว

2.7 อธิบายอัลกอริทึมที่สำคัญในโปรแกรม

1. **Delta Time Algorithm** อัลกอริทึมคำนวณระยะห่างของเวลาในแต่ละเฟรม (now - lastNanos) / 1000000000.0 เพื่อนำไปคูณกับความเร็วของแอนิเมชัน ทำให้ความเร็วในการเดินของลูกค้าหรือการนับถอยหลังของเกมมีความเสถียร ราบรื่น และไม่แปรผันตามความเร็วของ CPU

```
private void tick() {
    long now = System.nanoTime();
    double dt = (now - lastNanos) / 1_000_000_000.0;
    lastNanos = now;
    if (dt > 0.05) {
        dt = 0.05;
    }
    updateGame(dt);
    paintImmediately(0, 0, getWidth(), getHeight());
    Toolkit.getDefaultToolkit().sync();
}
```

2. **Collision Detection (Hitbox)** อัลกอริทึมตรวจสอบการชนกันผ่านเมธอด contains() ของคลาส Rectangle ใช้เช็คว่าพิกัดปัจจุบันของเมาส์ (mx, my) ตกอยู่ภายในพื้นที่กรอบสี่เหลี่ยม (Bounding Box) ของวัตถุหรือลูกค้าหรือไม่ เพื่อสร้างระบบปฏิสัมพันธ์ในเกม

```
public boolean contains(int mx, int my) {
    int dx = drawX();
    int dy = drawY();
    return mx >= dx && mx <= dx + DRAW_W && my >= dy && my <= dy + DRAW_H;
}
```

3. **Procedural Animation** การใช้สมการคณิตศาสตร์ฟังก์ชันตรีโกณมิติอย่าง Math.sin() สร้างแอนิเมชันแบบอัตโนมัติ เช่น การสร้างรอบก้าวเดินที่ดูเป็นธรรมชาติ (Math.sin(walkPhase)) หรือจังหวะการสั่นของตัวละครเมื่อโกรธ

```
public void update(double dt) {
    walkPhase += dt * 10 * walkSpeed;
    if (state == WALKING_IN) {
        x -= WALK_IN_SPEED * walkSpeed * dt;
        y = standY + Math.sin(walkPhase) * 5;
        bubbleScale = Math.min(1, bubbleScale + dt * 2.5);
        if (x <= targetX) {
            x = targetX;
            y = standY;
            state = WAITING;
            bubbleScale = 1;
        }
```

บทที่ 3 สรุปผล

3.1 ปัญหาที่พบระหว่างการพัฒนา

1. **ปัญหาการจัดการสัดส่วนภาพและ Hitbox (Sprite & Collision)** ในช่วงแรกพบปัญหาการโหลดไฟล์ภาพ .png ที่มีพื้นที่โปร่งใส (Transparent) รอบตัวละครกว้างไม่เท่ากัน ทำให้การเซ็ตค่าพิกัด Hitbox สำหรับให้เมาส์คลิกหรือลากวางทำงานผิดเพี้ยนไปจากที่คำนวณไว้ ต้องแก้ปัญหาด้วยการเขียนอัลกอริทึมตัดขอบภาพ (Content Box) และปรับสัดส่วนภาพด้วยตนเอง
2. **การจัดการสถานะตัวละครหลายตัว (State Machine Management)** ความท้าทายในการจัดการสถานะของลูกค้า 3 คนพร้อมกัน โดยลูกค้าแต่ละคนมี 5 สถานะ ได้แก่ เดินเข้า (WALKING_IN) รออาหาร (WAITING) ดีใจ (HAPPY) โกรธ (ANGRY) และเดินออก (WALKING_OUT) ซึ่งหากการจัดการเวลาใน Game Loop ลำดับผิดพลาด จะทำให้แอนิเมชันซ้อนทับกันหรือระบบคิดคะแนนทำงานผิดปกติ
3. **ปัญหาการจัดลำดับการวาดกราฟิก (Render Order)** เนื่องจากเกมใช้ Custom Rendering ทั้งหมด การเรียงลำดับโค้ด drawImage จึงสำคัญมาก หากเรียกวาดจานซูชิก่อนโต๊ะ ซูชิจะถูกโต๊ะทับหายไป จึงต้องวางโครงสร้างการวาดกราฟิกจากเลเยอร์หลังสุดมายังหน้าสุด อย่างระมัดระวัง

3.2 จุดเด่นของโปรแกรมที่ไม่เหมือนใคร

1. **ระบบปฏิสัมพันธ์แบบลากแล้ววาง (Custom Drag & Drop System)** เกมไม่ได้ใช้แค่การคลิกปุ่มเสิร์ฟแบบธรรมดา แต่ผู้เล่นต้องคลิกค้างที่จานซูชิแล้วลาก (Drag) ไปปล่อยลงบนตัวลูกค้าที่ต้องการ (Drop) พร้อมระบบไฮไลต์เป้าหมายเมื่อลากเมาส์ผ่าน ทำให้ได้ความรู้สึกเหมือนกำลังหยิบอาหารเสิร์ฟจริงๆ
2. **ระบบฟิสิกส์แอนิเมชันและเอฟเฟกต์อลังการ (Procedural Animation & FX)** มีการเขียนระบบ Particle ด้วยคลาส FX เองทั้งหมด เพื่อสร้างเอฟเฟกต์พลุกระจายและตัวเลขคะแนนลอยเด้งขึ้นมา นอกจากนี้ยังใช้สมการคณิตศาสตร์ (Math.sin) เพื่อสร้างแอนิเมชันการเดินโยกตัว อาการสั่นตอนลูกค้าโกรธ และการกระโดดดีใจ ทำให้เกมดูมีชีวิตชีวาและเป็นมืออาชีพ
3. **ระบบบริหารความเสี่ยงและคอมโบ (Dynamic Combo System)** เมื่อเลเวลสูงขึ้น ลูกค้าจะมีความอดทนน้อยลงเรื่อย ๆ แต่มาพร้อมระบบคอมโบ (Combo) ที่เพิ่มโบนัสคะแนนทุกครั้งที่เสิร์ฟถูกต่อเนื่อง และเมื่อครบ 5 คอมโบจะเข้าสู่ช่วง FEVER TIME ที่คะแนนเป็น 2 เท่า เป็นการกดดันให้ผู้เล่นต้องบริหารเวลาและรักษาสมาธิ
4. **ความหลากหลายของโหมดการเล่น ลูกค้า VIP และระบบบันทึกสถิติ (Game Modes, VIP & HighScore System)** ตัวเกมมีระบบ GameMode ที่เพิ่มความท้าทายหลากหลายรูปแบบ มีการสุ่มเกิดลูกค้าพิเศษ (VipCustomer) ที่สร้างความตื่นเต้นในการจัดลำดับความสำคัญของการเสิร์ฟ และมีระบบ HighScore คอยบันทึกสถิติสูงสุดเพื่อให้ผู้เล่นสามารถกลับมาเล่นซ้ำเพื่อทำลายสถิติเดิมของตัวเองได้

บทที่ 3 สรุปผล

3.3 คำแนะนำสำหรับผู้สอน

ผู้จัดทำขอขอบพระคุณอาจารย์ผู้สอนเป็นอย่างยิ่งที่ทุ่มเทให้กับการสอนอย่างเต็มที่เสมอมา อาจารย์ได้มอบทั้งความรู้และโจทย์แบบฝึกหัดที่หลากหลาย ซึ่งเป็นประโยชน์ต่อการพัฒนาทักษะของนักศึกษาเป็นอย่างมาก

อย่างไรก็ตาม ในมุมมองของผู้เรียน โดยเฉพาะกลุ่มนักศึกษาที่มีพื้นฐานด้านการเขียนโปรแกรมน้อย การทำความเข้าใจเนื้อหาในชั้นเรียนบางช่วงอาจมีความท้าทาย เนื่องจากจังหวะการอธิบายของอาจารย์ค่อนข้างรวดเร็ว ประกอบกับบางครั้งอาจมีข้อจำกัดเรื่องการรับฟังเสียงในห้องเรียน ทำให้ติดตามเนื้อหาหรือทำความเข้าใจตามไม่ทันในครั้งแรก จึงอยากเสนอแนะและขอความอนุเคราะห์จากอาจารย์สำหรับรุ่นต่อไป ดังนี้

1. การปรับจังหวะการอธิบายและทบทวนเนื้อหา: หากเป็นไปได้ อยากให้อาจารย์พิจารณาปรับจังหวะการอธิบายให้ช้าลงในหัวข้อที่ซับซ้อน และเพิ่มการทบทวน (Review) เนื้อหาที่สำคัญซ้ำอีกครั้ง เพื่อให้นักศึกษาสามารถย่อยข้อมูลและทำความเข้าใจตามได้ทัน
2. การอธิบายเจาะลึกถึงพื้นฐานของโค้ดและคำสั่ง: อยากให้อาจารย์ช่วยอธิบายปูพื้นฐานเจาะลึกถึงที่มาที่ไปของคำสั่งหรือเมธอด (Method) ต่างๆ ในโค้ดให้ช้าลงและละเอียดขึ้น เพื่อให้นักศึกษาเข้าใจแก่นแท้ว่าโค้ดแต่ละส่วนคืออะไรและทำงานอย่างไร แทนที่จะเป็นการพิมพ์ตามเพียงอย่างเดียว

ท้ายที่สุดนี้ ผู้จัดทำขอขอบพระคุณอาจารย์อีกครั้งจากใจจริง สำหรับความตั้งใจและความทุ่มเทในการถ่ายทอดความรู้ให้กับนักศึกษาตลอดภาคการศึกษาครับ